CSC 830 - Advanced Artificial Intelligence

Lecture 04: Goal Trees and Rule-Based Expert Systems

Prof. Nwojo Agwu Nnanna
nagwu@nileuniversity.edu.ng

March 3, 2025

Nile University of Nigeria, Abuja

Table of contents

Logical Reasoning Systems

Introduction

- Real knowledge representation and reasoning systems come in several major varieties.
- They are all based on FOL but departing from it in different ways
- These differ in their intended use, degree of formal semantics, expressive power, practical considerations, features, limitations, etc.
- Some major families of reasoning systems are
  - Theorem provers
  - Logic programming languages
  - Rule-based or production systems
  - Semantic networks
  - Frame-based representation languages
  - Databases (deductive, relational, object-oriented, etc.)
  - Constraint reasoning systems
  - Truth maintenance systems
  - Description logics

Rule-Based Expert (Production) Systems

- The notion of a "production system" (PS) was invented in 1943 by Post to describe re-write rules for symbol strings
- Used as the basis for many rule-based expert systems
- Most widely used KB formulation in practice
- A production is a rule of the form:

C1, C2, ... Cn => A1 A2 ...Am

Left hand side (LHS)
Conditions/antecedents

Condition which must hold before the rule can be applied

Right hand side (RHS)
Conclusion/consequence

Actions to be performed or conclusions to be drawn when the rule is applied

Three Basic Components of PS

- **Rule Base (database of rules, also called knowledge base)**
  - Unordered set of user-defined "if-then" rules.
  - Form of rules: if $P_1 \wedge \cdots \wedge P_m$ then $A_1, \ldots, A_n$ the P_is are conditions (often facts) that determine when rule is applicable.
  - Actions can add or delete facts from the Working Memory.
  - Example rule (in CLIPS format)

```
(defrule determine-gas-level
    (working-state engine does-not-start)
    (rotation-state engine rotates)
    (maintenance-state engine recent)
    ⇒ (assert (repair "Add gas.")))
```

- **Working Memory (WM) (a database of facts)** – A set of "facts", represented as literals, defining what's known to be true about the world
  - Often in the form of "flat tuples" (similar to predicates), e.g., (age Fred 45)
  - WM initially contains case specific data (not those facts that are always true in the world)
  - Inference may add/delete fact from WM
  - WM will be cleared when a case is finished
- **Inference Engine (also called an interpreter)** – Procedure for inferring changes (additions and deletions) to Working Memory.
  - Can be both forward and backward chaining
  - Usually a cycle of three phases: match, conflict resolution, and action, (in that order)

Basic Inference Procedure

While changes are made to Working Memory do:

- **Match** the current WM with the rule-base
  - Construct the Conflict Set – the set of all possible (rule, facts) pairs where rule is from the rule-base, facts from WM that unify with the conditional part (i.e., LHS) of the rule.
- **Conflict Resolution**: Instead of trying all applicable rules in the Conflict set, select one from the Conflict Set for execution. (**depth-first**)
- **Act/fire**: Execute the actions associated with the conclusion part of the selected rule, after making variable substitutions determined by unification during match phase
- **Stop** when conflict resolution fails to return any (rule, facts) pair

Conflict Resolution Strategies

- **Refraction**
  - A rule can only be used once with the same set of facts in WM. This strategy prevents firing a single rule with the same facts over and over again (avoiding loops)
- **Recency**
  - Use rules that match the facts that were added most recently to WM, providing a kind of "focus of attention" strategy.
- **Specificity**
  - Use the most specific rule,
  - If one rule's LHS is a superset of the LHS of a second rule, then the first one is more specific
  - If one rule's LHS implies the LHS of a second rule, then the first one is more specific
- **Explicit priorities**
  - E.g., select rules by their pre-defined order/priority
- **Precedence of strategies**

Example

R1: P(x) => Q(x); R2: Q(y) => S(y); WM = {P(a), P(b)}

conflict set: {(R1, P(a)), (R1, P(b))}

by rule order: apply R1 on P(a); WM = {Q(a), P(a), P(b)}

conflict set: {(R2, Q(a)), (R1, P(a)), (R1, P(b))}

by recency: apply R2 on Q(a) WM = {S(a), Q(a), P(a), P(

conflict set: {(R2, Q(a)), (R1, P(a)), (R1, P(b))}

by refraction, apply R1 on P(b): WM = {Q(b), S(a), Q(a), P(

conflict set: {(R2, Q(b)), (R2, Q(a)), (R1, P(a)), (R1, P(b))}

by recency, apply R2 on Q(b): WM = {S(b), Q(b), S(a), Q(

Default Reasoning

- Reasoning that draws a plausible inference on the basis of less than conclusive evidence in the absence of information to the contrary
  - If WM $= \{$bird(tweedy)$\}$, then by default, we can conclude that fly(tweedy)
  - When we also know that penguin(tweedy), then we should change the conclusion to $\sim$fly(tweedy)
  - Bird(x) $\Rightarrow$ fly(x) is a default rule (true in general, in most cases, almost)
  - Default reasoning is thus non-monotonic
  - Formal study of default reasons: default logic (Reiter), nonmonotonic logic (McDermott), circumscription (McCarthy) one conclusion: default reasoning is totally undecidable
  - Production system can handle simple default reasoning
    - By specificity: default rules are less specific
    - By rule priority: put default rules at the bottom of the rule base
    - Retract default conclusion (e.g., fly(tweedy)) is complicated

Other Issues

- PS can work in backward chaining mode
  - Match RHS with the goal statement to generate subgoals
  - Mycin: an expert system for diagnosing blood infectious diseases
- Expert system shell
  - A rule-based system with empty rule base
  - Contains data structure, inference procedures, AND user interface to help encode domain knowledge
  - Emycin (backward chaining) from Stanford U
  - OPP5 (forward chaining) from CMU and its descendants CLIPS, Jess.
- Metarules
  - Rules about rules
  - Specify under what conditions a set of rules can or cannot apply
  - For large, complex PS
- Consistency check of the rule-base is crucial (as in FOL)
- Uncertainty in PS (to be discussed later)

Comparing PS and FOL

- Advantages
  - Simplicity (both KR language and inference),
  - Inference more efficient
  - Modularity of knowledge (rules are considered, to a degree, independent of each other), easy to maintain and update
  - Similar to the way humans express their knowledge in many domains
  - Can handle simple default reasoning
- Disadvantages
  - No clearly defined semantics (may derive incorrect conclusions)
  - Inference is not complete (mainly due to the depth-first procedure)
  - Inference is sensitive to rule order, which may have unpredictable side effects
  - Less expressive (may not be suitable to some applications)
- No explicit structure among pieces of knowledge in BOTH FOL (a un-ordered set of clauses) and PS (a list of rules)

Problem-Solving Strategies

- In a rule-based system, the inference engine determines which rule antecedents, if any, are satisfied by the facts. Two general methods of inferring are commonly used as the problem-solving strategies of expert systems:
  1. forward-chaining
  2. backward-chaining.

Forward Chaining

- Forward chaining – deduction is used to reach a conclusion from a set of antecedents.
- Forward-chaining is reasoning from facts to the conclusions resulting from those facts. For example, if you see that it is raining before leaving home (the fact) then you should take an umbrella (the conclusion).
  - Given a new fact, generate all consequences
  - Assumes all rules are of the form

$$C_1 \wedge C_2 \wedge C_3 \wedge C_n \Rightarrow \text{ Result}$$

  - Each rule and binding generates a new fact
  - This new fact will "trigger" other rules
  - Keep going until the desired fact is generated

- Is known as **data-driven reasoning** – reasoning starts from a set of data and ends up at the goal (conclusion).
- First, take facts in the fact database and determine if any combination of these matches all the antecedents of one of the rules in the rule base.
  - A rule is **triggered** when all its antecedents are matched by facts in the database.
  - When a rule is triggered, it is then **fired** – that is, its conclusion is added to the facts database.

The Forward-Chaining Algorithm

function FOL-FC-ASK(KB, α) **returns** a substitution or *false*

repeat until *new* is empty
  $new \leftarrow \{\}$
  for each sentence r **in** KB **do**
    $(p_1 \wedge \ldots \wedge p_n \Rightarrow q) \leftarrow$ STANDARDIZE-APART(r)
    for each θ such that $(p_1 \wedge \ldots \wedge p_n)\theta = (p'_1 \wedge \ldots \wedge p'_n)\theta$
        for some $p'_1, \ldots, p'_n$ in KB
      $q' \leftarrow$ SUBST(θ, q)
      if q' is not a renaming of a sentence already in KB or new **then do**
        add q' to new
        $\phi \leftarrow$ UNIFY(q', α)
        if ϕ is not *fail* **then return** ϕ
  add new to KB
return *false*

Backward Chaining

- Forward chaining
  - Applies to a set of rules and facts to deduce conclusions
  - Useful when a set of facts are present but one does not know what conclusions one is trying to prove
  - May be inefficient because it may end up proving a number of conclusions
    - In such cases, where a single specific conclusion is to be proved, backward chaining is more appropriate
- Backward chaining – starts from a conclusion, which is the hypothesis we wish to prove, and we aim to show that the conclusion can be reached from the rules and facts in the database.
- The conclusion to prove is called a goal, and therefore reasoning in this way is known as goal-driven reasoning.

- Backward-chaining involves reasoning in the reverse from a hypothesis, a potential conclusion to be proved, to the facts that support the hypothesis.
- For example, if you have not looked outside and someone enters with wet shoes and an umbrella, your hypothesis will be that it is raining. In order to support this hypothesis, you could ask the person if it was, in fact, raining. If the response is yes, then the hypothesis is proved true and becomes a fact.

- Consider the item to be proven a goal
- Find a rule whose head is the goal (and bindings)
- Apply bindings to the body, and prove these (subgoals) in turn
- If you prove all the subgoals, increasing the binding set as you go, you will prove the item.

function FOL-BC-ASK(KB, goals, θ) **returns** a set of substitutions

inputs: KB, a knowledge base

$goals$, a list of conjuncts forming a query

θ, the current substitution, initially the empty substitution $\{\}$

local variables: ans, a set of substitutions, initially empty

if $goals$ is empty **then return** $\{\theta\}$

$q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(goals))$

for each r **in** KB where $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \ldots \wedge p_n \Rightarrow q)$

and $\theta' \leftarrow \text{UNIFY}(q, q')$ succeeds

$ans \leftarrow \text{FOL-BC-ASK}(KB, [p_1, \ldots, p_n | \text{REST}(goals)], \text{COMPOSE}(\theta', \theta)) \cup ans$

return ans

References

[1] Pang-Ning Tan, Michael Steinbach, Vipin Kumar, *Introduction to Data Mining*, Addison Wesley 2006

[2] Coppin, B., *Artificial Intelligence Illuminated,* Jones and Bartlett, 2004.

[3] Luger, G. F., *Artificial Intelligence,*Addison Wesley, 2005.

[4] Russell, S., and Norvig, P., *Artificial Intelligence: A Modern Approach,* Prentice Hall.

[5] Winston, P. H., *Artificial Intelligence,* Addison Wesley.